

Java 骰子模擬器程式碼

https://claude.ai/code/session_019TTkqekVWyx36dkK3PTeK (Claude)

User prompt: (附上題目截圖 Exercise 1 「骰子模擬器」規格)

接下來可以做骰子這題，把題目截圖和你的問題傳給我，我們用同樣的方式跑一次真實對話，再整理出 `hw2.java`、PDF 和 reflection。並直接上傳到我的 github

Exercise 1

題目：實作一個「骰子模擬器」。

規格：

- 視窗標題「骰子模擬器」，尺寸 400x320，關閉時結束程式，開啟時置中
- 中央有一個 JLabel 顯示目前點數，字體 60pt
- 下方一個「擲骰子」按鈕
- 每按一次隨機產生 1-6 的點數並更新顯示
- 視窗上方顯示「已擲 N 次，總和 M，平均 X.XX」
- 點數為 6 時文字變綠色，為 1 時變紅色，其餘黑色

Response: 好，這題我直接照規格寫一支骰子模擬器。這段問答一樣會收進 PDF，作為 AI 對話紀錄。

怎麼對應規格

- 視窗：標題「骰子模擬器」、400×320，用 EXIT_ON_CLOSE 讓關閉時結束程式，setLocationRelativeTo(null) 讓視窗開啟時置中。
- 版面：用 BorderLayout 排成上、中、下三塊。上方（NORTH）是統計文字，中央（CENTER）是點數，下方（SOUTH）是按鈕。
- 點數：一個 JLabel，字體 60pt，水平置中。還沒擲之前顯示「-」，不放數字，免得和真的點數搞混。
- 擲骰子：用 Random.nextInt(6) + 1 產生 1-6 的點數，同時更新次數 N 和總和 M。
- 平均：用 String.format("%.2f", ...) 輸出到小數點後兩位。總和要先轉成 double 再除，否則會變成整數除法，小數被捨掉。
- 顏色：點數是 6 變綠色，是 1 變紅色，其他點數是黑色。

程式碼

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class hw2 extends JFrame {

    private final JLabel statsLabel = new JLabel("已擲 0 次，總和 0，平均 0.00", SwingConstants.CENTER);
    private final JLabel diceLabel = new JLabel("-", SwingConstants.CENTER);
    private final JButton rollButton = new JButton("擲骰子");
    private final Random random = new Random();

    private int count = 0;    // 已擲次數 N
    private int sum = 0;      // 點數總和 M

    public hw2() {
        setTitle("骰子模擬器");
        setSize(400, 320);
```

```

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null);
setLayout(new BorderLayout());

statsLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 16));
statsLabel.setBorder(BorderFactory.createEmptyBorder(10, 0, 0, 0));
diceLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 60));

JPanel bottom = new JPanel();
bottom.add(rollButton);

add(statsLabel, BorderLayout.NORTH);
add(diceLabel, BorderLayout.CENTER);
add(bottom, BorderLayout.SOUTH);

rollButton.addActionListener(e -> roll());

setVisible(true);
}

private void roll() {
    int value = random.nextInt(6) + 1;    // 1 ~ 6
    count++;
    sum += value;

    diceLabel.setText(String.valueOf(value));
    if (value == 6) {
        diceLabel.setForeground(new Color(0, 150, 0));    // 綠色
    } else if (value == 1) {
        diceLabel.setForeground(Color.RED);
    } else {
        diceLabel.setForeground(Color.BLACK);
    }

    double avg = (double) sum / count;
    statsLabel.setText(String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, avg));
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(hw2::new);
}
}

```

綠色我用了 `new Color(0, 150, 0)`，沒有用 `Color.GREEN`。 `Color.GREEN` 是很亮的螢光綠，在灰白背景上不太容易看清楚。

接下來我會實際執行測試，再做 PDF、寫 reflection，然後 push 上去。

第二輪：基本觀念提問

User prompt: 根據我個人的這 12 個提問，請參照類似的格式給我對應的回答後製作成 PDF

HW1：登入視窗

1. `JFrame` 是什麼？為什麼我的類別要寫 `extends JFrame`？
2. `setLayout(null)` 是什麼意思？跟 `FlowLayout`、`BorderLayout` 有什麼差別？
3. `setBounds(x, y, 寬, 高)` 的四個數字分別代表什麼？座標原點在哪裡？
4. 為什麼 `setVisible(true)` 要放在所有元件都加完之後？
5. Java 的字串為什麼不能用 `==` 比較？`==` 和 `.equals()` 差在哪？
6. `JTextField` 和 `JPasswordField` 有什麼不同？為什麼 `getPassword()` 回傳的是 `char[]`，不是 `String`？
7. `addActionListener(e -> ...)` 這種箭頭寫法 (lambda) 是什麼？不用 lambda 的話要怎麼寫？
8. `System.out.println` 和 `JOptionPane.showMessageDialog` 的差別是什麼？各自適用在什麼情況？
9. `setDefaultCloseOperation` 有哪幾種選項？沒有設定的話，按 X 會發生什麼事？
10. `SwingUtilities.invokeLater` 是做什麼的？什麼是事件分派執行緒 (EDT)？
11. 為什麼把帳號密碼直接寫在程式碼裡不安全？實務上通常會怎麼處理？
12. `getText().trim()` 裡的 `trim()` 有什麼用？如果不加，使用者輸入時會遇到什麼問題？

HW2：骰子模擬器

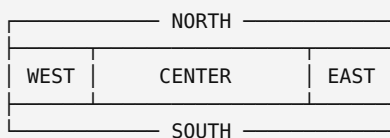
1. `BorderLayout` 的 NORTH、CENTER、SOUTH、EAST、WEST 是怎麼分配空間的？
2. 為什麼按鈕要先放進 `JPanel`，再加入到 SOUTH？直接加會怎樣？
3. `Random.nextInt(6) + 1` 為什麼能產生 1 到 6？`nextInt(6)` 本身的範圍是多少？
4. `Math.random()` 和 `Random` 類別有什麼差別？
5. `(double) sum / count` 為什麼要先轉成 `double`？不轉的話平均會算錯多少？
6. `String.format("%.2f", ...)` 裡的 `%d`、`%.2f` 這些格式符號各代表什麼？
7. `new Font(名稱, 樣式, 大小)` 的三個參數怎麼設定？60pt 在螢幕上大概多大？
8. `setForeground` 和 `setBackground` 差在哪？`Color` 可以怎麼自訂顏色？
9. `count` 和 `sum` 為什麼要宣告成類別的成員變數，不能寫在 `roll()` 方法裡面？
10. `SwingConstants.CENTER` 是做什麼的？怎麼讓文字同時水平和垂直置中？
11. 如果要加一個「重設」按鈕，讓次數、總和和顯示都歸零，程式要怎麼改？
12. 怎麼改成一次擲兩顆骰子，並顯示兩顆的點數和總和？

(同一則提問同時問了兩個程式，這份紀錄收錄的是回答中「HW2：骰子模擬器」的部分)

Response:

1. BorderLayout 的 NORTH、CENTER、SOUTH、EAST、WEST 是怎麼分配空間的？

- **NORTH/SOUTH**：寬度撐滿整個視窗，高度用元件「剛好需要」的高度。
- **EAST/WEST**：高度是扣掉上下之後剩下的高度，寬度用元件剛好需要的寬度。
- **CENTER**：拿走剩下的所有空間。視窗放大時，多出來的空間全部給 CENTER。
- 沒放東西的區域不佔任何空間。
- 每個區域只能放一個元件，同一個區域 add 第二次，會把前一個換掉。
- 所以骰子模擬器把統計文字放上面、點數放中間、按鈕放下面，點數會自動佔據中間最大的那一塊。



2. 為什麼按鈕要先放進 JPanel，再加入到 SOUTH？直接加會怎樣？

- 照上一題的規則，放在 SOUTH 的元件**寬度會被撐滿**整個視窗。按鈕直接加到 SOUTH，就會變成一條跟視窗一樣寬的長條按鈕，看起來很怪。
- JPanel 預設是 `FlowLayout`，會讓元件維持原本的大小，並在一列中置中。所以先把按鈕放進 `JPanel`，再把 `JPanel` 放到 SOUTH：被撐滿的是 `JPanel`，按鈕本身維持正常大小、乖乖待在中間。
- 另一個好處是之後要再加按鈕（例如第 11 題的「重設」）很方便，直接 `bottom.add()` 就排在旁邊。

3. `Random.nextInt(6) + 1` 為什麼能產生 1 到 6？`nextInt(6)` 本身的範圍是多少？

- `nextInt(n)` 會回傳 0 到 `n-1` 之間的整數，每個數字機率相同。包含 0，不包含 `n`。
- 所以 `nextInt(6)` 會是 0、1、2、3、4、5 其中一個，`+1` 之後就變成 1~6。
- 常見錯誤：
 - 寫 `nextInt(6)` 忘了 `+1`，會擲出 0，而且永遠擲不出 6。
 - 寫成 `nextInt(7)`，範圍變成 0~6，還是會出現 0。
- Java 17 之後可以直接指定範圍：`random.nextInt(1, 7)`，一樣是包含 1、不包含 7。

4. `Math.random()` 和 `Random` 類別有什麼差別？

- `Math.random()`：靜態方法，直接呼叫就好，回傳 0.0（含）到 1.0（不含）之間的 `double`。要變成骰子點數得自己換算：

```
int value = (int) (Math.random() * 6) + 1;
```

- `Random`：要先 `new` 一個物件，但方法比較多：`nextInt()`、`nextDouble()`、`nextBoolean()` 等，產生整數不用自己換算。
- 可以指定種子（seed）：`new Random(42)` 每次跑出來的序列都一樣，寫測試或重現 bug 時很好用。
- 其實 `Math.random()` 內部就是用一個共用的 `Random` 物件做的。
- 兩者都是「偽隨機」，用公式算出來、看起來隨機而已，不適合拿來產生密碼或驗證碼；那種情況要用 `SecureRandom`。

5. `(double) sum / count` 為什麼要先轉成 `double`？不轉的話平均會算錯多少？

- Java 裡整數除以整數，結果還是整數，小數部分直接捨去（不是四捨五入）。
- 例子：擲兩次，點數 3 和 4，總和 7：

```
7 / 2           // 3      → 顯示 3.00（錯）
(double) 7 / 2 // 3.5    → 顯示 3.50（對）
```

- 最多會少算將近 1（例如真正的平均是 3.99，卻顯示 3.00）。
- 注意轉型的位置：`(double)` 的優先順序比除法高，所以 `(double) sum / count` 是「先把 `sum` 轉成 `double` 再除」。如果寫成 `(double) (sum / count)`，就變成先做整數除法、捨去小數之後才轉型，結果一樣是錯的。

6. `String.format("%.2f", ...)` 裡的 `%d`、`%.2f` 這些格式符號各代表什麼？

符號	意思	範例
<code>%d</code>	整數	<code>String.format("%d", 15)</code> → 15
<code>%f</code>	小數，預設 6 位	3.5 → 3.500000
<code>%.2f</code>	小數，取到第 2 位（四捨五入）	3.456 → 3.46
<code>%s</code>	字串	"骰子" → 骰子
<code>%5d</code>	整數，至少佔 5 格寬	42 → 42
<code>%%</code>	印出 <code>%</code> 符號本身	<code>%</code>

- 後面的參數會依照順序填進前面的符號：

```
String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, avg);
//           ↑ count      ↑ sum      ↑ avg
```

- 型別要對得上，例如把 `double` 丟給 `%d` 會在執行時出錯。

7. new Font(名稱, 樣式, 大小) 的三個參數怎麼設定？60pt 在螢幕上大概多大？

- 名稱：
 - 邏輯字型：Font.SANS_SERIF（無襯線）、Font.SERIF（有襯線）、Font.MONOSPACED（等寬）、Font.DIALOG。Java 會自動對應到系統上的字型，在每台電腦都能用。
 - 也可以寫實際字型名稱，例如 "Microsoft JhengHei"（微軟正黑體），但對方電腦沒裝這個字型時就會被換成預設字型。
- 樣式：Font.PLAIN（一般）、Font.BOLD（粗體）、Font.ITALIC（斜體），也可以組合 Font.BOLD | Font.ITALIC。
- 大小：單位是點（pt）。
- 60pt 有多大：Java 在螢幕上大致把 1pt 當成 1 像素，所以在 100% 縮放的螢幕上，60pt 的字大約 60 像素高。如果 Windows 的顯示縮放設在 150%，就會再放大成大約 90 像素。實際看起來大概是一兩公分高，在 400×320 的視窗裡算很醒目。

8. setForeground 和 setBackground 差在哪？Color 可以怎麼自訂顏色？

- setForeground：前景色，也就是文字的顏色。骰子的紅、綠、黑就是用這個。
- setBackground：背景色。
 - 陷阱：JLabel 預設是透明的，直接 setBackground 看不出任何變化，要先 setOpaque(true) 背景才會畫出來。
- 自訂 Color 的方法：

```
Color.RED                // 內建常數
new Color(0, 150, 0)     // RGB，每個值 0~255
new Color(0, 150, 0, 128) // 第 4 個參數是透明度
Color.decode("#2E7D32")  // 用網頁色碼
```

9. count 和 sum 為什麼要宣告成類別的成員變數，不能寫在 roll() 方法裡面？

- 寫在方法裡的可叫區域變數，每次呼叫方法時重新建立，方法結束就消失。
- 如果寫在 roll() 裡面，每按一次按鈕都會從 0 開始：

```
private void roll() {
    int count = 0; // 每次都重新歸零
    count++;      // 永遠只會是 1
}
```

畫面就會永遠顯示「已擲 1 次」，總和也只是這一次的點數。

- 成員變數（欄位）屬於物件本身，只要視窗物件還在，它的值就會一直保留，所以能在多次點擊之間累加。這也是物件導向的核心概念：物件會記住自己的狀態。

10. SwingConstants.CENTER 是做什麼的？怎麼讓文字同時水平和垂直置中？

- SwingConstants.CENTER 是一個代表「置中」的常數。放在 JLabel 建構子的第二個參數，是設定水平對齊：

```
new JLabel("-", SwingConstants.CENTER); // 水平置中
```

- JLabel 的水平對齊預設是靠左（LEADING），所以要特別設定；垂直對齊預設就已經是置中。要明確設定的話可以用：

```
label.setHorizontalAlignment(SwingConstants.CENTER);
label.setVerticalAlignment(SwingConstants.CENTER);
```

- 在骰子模擬器裡，點數 JLabel 放在 BorderLayout.CENTER，標籤本身會被撐滿中間整塊區域，再加上水平、垂直都置中，數字就會剛好在正中央。

11. 如果要加一個「重設」按鈕，讓次數、總和和顯示都歸零，程式要怎麼改？

步驟：

1. 建立一個「重設」按鈕，加進下方的 `JPanel`，跟「擲骰子」並排。
2. 寫一個 `reset()` 方法，把 `count`、`sum` 歸零，點數改回「-」、顏色改回黑色，再更新統計文字。
3. 把更新統計文字的程式抽成 `updateStats()`，讓 `roll()` 和 `reset()` 共用。

```
// 建構子裡
JButton resetButton = new JButton("重設");
JPanel bottom = new JPanel();
bottom.add(rollButton);
bottom.add(resetButton);
resetButton.addActionListener(e -> reset());

// 新增的方法
private void reset() {
    count = 0;
    sum = 0;
    diceLabel.setText("-");
    diceLabel.setForeground(Color.BLACK);
    updateStats();
}

private void updateStats() {
    double avg = (count == 0) ? 0 : (double) sum / count;
    statsLabel.setText(String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, avg));
}
```

- 注意 `count == 0` 的判斷：歸零之後如果直接算 `sum / count`，`double` 的除法不會當掉，但會得到 `NaN`（不是數字），畫面會顯示「平均 NaN」。所以次數是 0 的時候，平均直接給 0。

12. 怎麼改成一次擲兩顆骰子，並顯示兩顆的點數和總和？

先決定規格：

- 「擲一次」等於同時擲兩顆，次數 N 加 1。
- 總和 M 累加兩顆的合計，平均是「每次兩顆合計」的平均（範圍 2~12）。
- 顏色規則套用在每一顆骰子上：6 綠、1 紅、其他黑。

畫面上用 `GridLayout(1, 2)` 讓兩顆骰子左右並排，下面再放一個顯示合計的標籤：

```
private final JLabel dice1 = new JLabel("-", SwingConstants.CENTER);
private final JLabel dice2 = new JLabel("-", SwingConstants.CENTER);
private final JLabel totalLabel = new JLabel("合計：-", SwingConstants.CENTER);

// 建構子裡（取代原本的 add(diceLabel, BorderLayout.CENTER)）
JPanel diceRow = new JPanel(new GridLayout(1, 2));
dice1.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 60));
dice2.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 60));
diceRow.add(dice1);
diceRow.add(dice2);

JPanel center = new JPanel(new BorderLayout());
center.add(diceRow, BorderLayout.CENTER);
center.add(totalLabel, BorderLayout.SOUTH);
add(center, BorderLayout.CENTER);

// 擲骰子
private void roll() {
    int a = random.nextInt(6) + 1;
    int b = random.nextInt(6) + 1;
    int total = a + b;
    count++;
    sum += total;

    showDie(dice1, a);
    showDie(dice2, b);
    totalLabel.setText("合計：" + total);

    double avg = (double) sum / count;
```

```
statsLabel.setText(String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, avg));
}

// 兩顆骰子共用的顯示方法
private void showDie(JLabel label, int value) {
    label.setText(String.valueOf(value));
    if (value == 6) label.setForeground(new Color(0, 150, 0));
    else if (value == 1) label.setForeground(Color.RED);
    else label.setForeground(Color.BLACK);
}
```

- 把「顯示一顆骰子」抽成 `showDie()` 方法，兩顆共用同一段邏輯，不用把顏色判斷寫兩遍。之後要改成三顆、四顆也很容易擴充。
- 補充：兩顆骰子合計的分布**不是平均的**，7 出現的機率最高（6/36），2 和 12 最低（各 1/36）。擲很多次之後，平均會接近 7。